EJERCICIO. Gestion de bases de datos-V1

Vamos a realizar un ejercicio que consiste en una aplicación WIN FORMS que permite gestionar una base de datos.

Ejercicio WinForms + SQL Server (CRUD) con separación Modelo-Vista-Controlador

Tabla: Provincias (idprovincia, provincia) | BD: bdpruebas01 | Servidor: Local

1. Objetivo del ejercicio

Vamos a construir una aplicación WinForms muy sencilla que permita gestionar (CRUD) la tabla Provincias en SQL Server:

- Acceder a la tabla Provincias.
- Mostrar los registros en una tabla (DataGridView).
- Insertar un nuevo registro.
- Modificar un registro existente.
- Eliminar un registro.

Para aprender bien, separaremos responsabilidades con una estructura inspirada en MVC:

- Modelo (Model): clases de datos (POCO).
- Acceso a datos (Repository/Data): ADO.NET (SqlConnection, SqlCommand).
- Controlador / Servicio (Controller/Service): reglas de negocio y coordinación.
- Vista (View): el formulario WinForms (solo UI y llamadas al controlador).

2. Preparación de la base de datos

Si ya tienes la BD y la tabla creadas, puedes saltar este apartado. Si no, ejecuta este script en SQL Server Management Studio (SSMS).

2.1 Script SQL (crear BD y tabla)

```
-- 1) Crear la base de datos (si no existe)
IF DB_ID('bdpruebas01') IS NULL
BEGIN
    CREATE DATABASE bdpruebas01;
END
GO

USE bdpruebas01;
GO
```

```
-- 2) Crear tabla Provincias (si no existe)
IF OBJECT_ID('dbo.Provincias', 'U') IS NULL
BEGIN
    CREATE TABLE dbo.Provincias
    (
        idprovincia INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  -- Clave primaria autoincremental
        provincia   NVARCHAR(100) NOT NULL                   -- Nombre de la provincia
    );
END
GO

-- 3) Datos de prueba (opcional)
IF NOT EXISTS (SELECT 1 FROM dbo.Provincias)
BEGIN
    INSERT INTO dbo.Provincias (provincia) VALUES
    ('Madrid'), ('Barcelona'), ('Valencia');
END
GO
```

2.2 Comprobar el servidor Local

Cuando dices “Servidor: Local”, normalmente puede ser una de estas opciones en SQL Server:

- . (punto) -> significa “este equipo” (instancia por defecto).
- .\SQLEXPRESS -> si instalaste SQL Server Express con esa instancia.
- (localdb)\MSSQLLocalDB -> si estás usando LocalDB (muy común en equipos de desarrollo).

En este ejercicio te dejo una conexión preparada para (localdb) y otra para .\SQLEXPRESS. Elige la que te funcione.

3. Crear el proyecto WinForms y la estructura de carpetas

3.1 Crear proyecto

1. Visual Studio -> Crear nuevo proyecto.
2. Selecciona: “Windows Forms App” (si es .NET Framework) o “Windows Forms App” (.NET) si trabajas con .NET 6/7/8.
3. Nombre sugerido: ProvinciasApp

3.2 Estructura (MVC adaptado a WinForms)

Dentro del proyecto, crea estas carpetas (clic derecho en el proyecto -> Add -> New Folder):

```
ProvinciasApp
│
├─ Models
│   └─ Provincia.cs
│
├─ Data
│   ├─ DbConfig.cs
│   └─ ProvinciaRepository.cs
│
├─ Controllers
```

```
│   └─ ProvinciasController.cs
│
└─ Views
    └─ FormProvincias.cs  (puede ser Form1 renombrado)
```

4. Modelo (Models)

El Modelo es una clase simple que representa una fila de la tabla Provincias.

4.1 Models/Provincia.cs

```
using System;

namespace ProvinciasApp.Models
{
    /// <summary>
    /// Representa una provincia (una fila de la tabla Provincias).
    /// </summary>
    public class Provincia
    {
        /// <summary>
        /// Identificador (PK) de la provincia.
        /// </summary>
        public int IdProvincia { get; set; }

        /// <summary>
        /// Nombre de la provincia.
        /// </summary>
        public string Nombre { get; set; } = string.Empty;
    }
}
```

5. Configuración de conexión (Data)

Centralizamos la cadena de conexión en una única clase para no repetirla por todo el proyecto.

5.1 Data/DbConfig.cs

```
namespace ProvinciasApp.Data
{
    /// <summary>
    /// Configuración de base de datos.
    /// Aquí guardamos la cadena de conexión (connection string).
    /// </summary>
    public static class DbConfig
    {
        // Opción A: LocalDB (muy común en PCs de desarrollo)
        // public static string ConnectionString =
        //
@"Server=(localdb)\MSSQLLocalDB;Database=bdpruebas01;Trusted_Connection=True;TrustServerCertificat
e=True;";

        // Opción B: SQL Express (instancia típica)
        public static string ConnectionString =
```

```
@"Server=.\SQLEXPRESS;Database=bdpruebas01;Trusted_Connection=True;TrustServerCertificate=True;";
    }
}
```

Notas rápidas:

- Trusted_Connection=True significa “usa autenticación de Windows”.
- TrustServerCertificate=True evita problemas de certificado en local.
- Si tu instancia es “.” (por defecto), prueba: Server=.;Database=bdpruebas01;...

6. Acceso a datos (Repository)

El Repository se encarga de hablar con SQL Server. Usaremos ADO.NET para que entiendas claramente qué ocurre en cada paso.

6.1 Data/ProvinciaRepository.cs (CRUD completo)

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.SqlClient;
using ProvinciasApp.Models;

namespace ProvinciasApp.Data
{
    /// <summary>
    /// Repository de Provincias: contiene toda la lógica de acceso a datos (SQL).
    /// IMPORTANTE: aquí NO debe haber código de interfaz gráfica (WinForms).
    /// </summary>
    public class ProvinciaRepository
    {
        private readonly string _connectionString;

        /// <summary>
        /// Recibe la cadena de conexión por constructor (fácil de cambiar y probar).
        /// </summary>
        public ProvinciaRepository(string connectionString)
        {
            _connectionString = connectionString;
        }

        /// <summary>
        /// Obtiene todas las provincias (SELECT).
        /// </summary>
        public List<Provincia> GetAll()
        {
            // Lista donde devolveremos los resultados
            List<Provincia> provincias = new List<Provincia>();

            // SqlConnection representa la conexión con SQL Server
            using (SqlConnection cn = new SqlConnection(_connectionString))
            {
                // La consulta SQL (ojo: no usamos * por buena práctica)
                string sql = "SELECT idprovincia, provincia FROM dbo.Provincias ORDER BY
```

```
provincia;";

                // SqlCommand representa “la orden” que enviaremos a SQL Server
                using (SqlCommand cmd = new SqlCommand(sql, cn))
                {
                    // Abrimos la conexión
                    cn.Open();

                    // Ejecutamos y obtenemos un lector (stream de filas)
                    using (SqlDataReader dr = cmd.ExecuteReader())
                    {
                        // Leemos fila a fila
                        while (dr.Read())
                        {
                            // Creamos un objeto Provincia por cada fila
                            Provincia p = new Provincia();

                            // Leemos campos por nombre (más claro)
                            p.IdProvincia = Convert.ToInt32(dr["idprovincia"]);
                            p.Nombre = Convert.ToString(dr["provincia"]) ?? "";

                            // Añadimos a la lista
                            provincias.Add(p);
                        }
                    }
                }
            }

            return provincias;
        }

        /// <summary>
        /// Inserta una provincia (INSERT) y devuelve el ID generado.
        /// </summary>
        public int Insert(Provincia provincia)
        {
            using (SqlConnection cn = new SqlConnection(_connectionString))
            {
                // SCOPE_IDENTITY() devuelve el último identity insertado en esta conexión
                string sql = @"
INSERT INTO dbo.Provincias (provincia)
VALUES (@provincia);
SELECT CAST(SCOPE_IDENTITY() AS INT);";

                using (SqlCommand cmd = new SqlCommand(sql, cn))
                {
                    // Parámetro para evitar SQL Injection y problemas de comillas
                    cmd.Parameters.AddWithValue("@provincia", provincia.Nombre);

                    cn.Open();

                    // ExecuteScalar devuelve la primera columna de la primera fila del resultado
                    int nuevoId = (int)cmd.ExecuteScalar();

                    return nuevoId;
                }
            }
```

```
        }

        /// <summary>
        /// Actualiza una provincia (UPDATE).
        /// Devuelve true si se actualizó 1 fila; false si no se encontró.
        /// </summary>
        public bool Update(Provincia provincia)
        {
            using (SqlConnection cn = new SqlConnection(_connectionString))
            {
                string sql = @"
UPDATE dbo.Provincias
SET provincia = @provincia
WHERE idprovincia = @id;";

                using (SqlCommand cmd = new SqlCommand(sql, cn))
                {
                    cmd.Parameters.AddWithValue("@provincia", provincia.Nombre);
                    cmd.Parameters.AddWithValue("@id", provincia.IdProvincia);

                    cn.Open();

                    // ExecuteNonQuery devuelve cuántas filas fueron afectadas
                    int filas = cmd.ExecuteNonQuery();

                    return filas == 1;
                }
            }
        }

        /// <summary>
        /// Elimina una provincia por ID (DELETE).
        /// Devuelve true si se eliminó 1 fila; false si no se encontró.
        /// </summary>
        public bool Delete(int idProvincia)
        {
            using (SqlConnection cn = new SqlConnection(_connectionString))
            {
                string sql = "DELETE FROM dbo.Provincias WHERE idprovincia = @id;";

                using (SqlCommand cmd = new SqlCommand(sql, cn))
                {
                    cmd.Parameters.AddWithValue("@id", idProvincia);

                    cn.Open();

                    int filas = cmd.ExecuteNonQuery();

                    return filas == 1;
                }
            }
        }

        /// <summary>
        /// Método extra: devuelve un DataTable (útil para enlazar directo a DataGridView).
        /// Alternativa a GetAll() si prefieres trabajar con DataTable.
        /// </summary>
```

```
        public DataTable GetAllAsDataTable()
        {
            DataTable tabla = new DataTable();

            using (SqlConnection cn = new SqlConnection(_connectionString))
            {
                string sql = "SELECT idprovincia, provincia FROM dbo.Provincias ORDER BY
provincia;";

                using (SqlDataAdapter da = new SqlDataAdapter(sql, cn))
                {
                    // Fill rellena el DataTable con el resultado del SELECT
                    da.Fill(tabla);
                }
            }

            return tabla;
        }
    }
}
```

7. Controlador / Reglas de negocio (Controllers)

El controlador es el “puente” entre la Vista (Form) y los Datos (Repository). Aquí validamos reglas: por ejemplo, no permitir nombres vacíos.

7.1 Controllers/ProvinciasController.cs

```
using System;
using System.Collections.Generic;
using System.Data;
using ProvinciasApp.Data;
using ProvinciasApp.Models;

namespace ProvinciasApp.Controllers
{
    /// <summary>
    /// Controlador de Provincias: reglas de negocio + llamadas al repositorio.
    /// </summary>
    public class ProvinciasController
    {
        private readonly ProvinciaRepository _repo;

        public ProvinciasController(ProvinciaRepository repo)
        {
            _repo = repo;
        }

        /// <summary>
        /// Devuelve todas las provincias (como lista).
        /// </summary>
        public List<Provincia> Listar()
        {
```

```
        return _repo.GetAll();
    }

    /// <summary>
    /// Devuelve todas las provincias (como DataTable) para DataGridView.
    /// </summary>
    public DataTable ListarTabla()
    {
        return _repo.GetAllAsDataTable();
    }

    /// <summary>
    /// Crea una provincia nueva, validando antes.
    /// </summary>
    public int Crear(string nombreProvincia)
    {
        // Regla 1: no permitir vacío
        if (string.IsNullOrWhiteSpace(nombreProvincia))
            throw new ArgumentException("La provincia no puede estar vacía.");

        // Regla 2 (opcional): recortar espacios
        nombreProvincia = nombreProvincia.Trim();

        Provincia p = new Provincia
        {
            Nombre = nombreProvincia
        };

        return _repo.Insert(p);
    }

    /// <summary>
    /// Modifica una provincia existente.
    /// </summary>
    public bool Modificar(int idProvincia, string nombreProvincia)
    {
        if (idProvincia <= 0)
            throw new ArgumentException("Id inválido.");

        if (string.IsNullOrWhiteSpace(nombreProvincia))
            throw new ArgumentException("La provincia no puede estar vacía.");

        nombreProvincia = nombreProvincia.Trim();

        Provincia p = new Provincia
        {
            IdProvincia = idProvincia,
            Nombre = nombreProvincia
        };

        return _repo.Update(p);
    }

    /// <summary>
    /// Elimina una provincia por ID.
    /// </summary>
    public bool Eliminar(int idProvincia)
```

```
        {
            if (idProvincia <= 0)
                throw new ArgumentException("Id inválido.");

            return _repo.Delete(idProvincia);
        }
    }
}
```

8. Vista (WinForms)

La Vista (Form) contiene controles y eventos (botones, grid, etc.). Su trabajo es:

- Leer datos de la pantalla (TextBox, fila seleccionada del grid...).
- Llamar al controlador.
- Mostrar resultados o errores con mensajes amigables.

8.1 Diseño del formulario

En el diseñador de WinForms crea un formulario con estos controles (nombres sugeridos):

- DataGridView: dgvProvincias
- TextBox: txtProvincia
- TextBox (solo lectura): txtId (para ver el id seleccionado)
- Button: btnCargar (texto: Cargar)
- Button: btnInsertar (texto: Insertar)
- Button: btnModificar (texto: Modificar)
- Button: btnEliminar (texto: Eliminar)
- Button: btnLimpiar (texto: Limpiar)

Consejo: pon txtId como ReadOnly=True para evitar que el usuario escriba el ID a mano.

8.2 Código del formulario (Views/FormProvincias.cs)

```
using System;
using System.Data;
using System.Windows.Forms;
using ProvinciasApp.Controllers;
using ProvinciasApp.Data;

namespace ProvinciasApp.Views
{
    public partial class FormProvincias : Form
    {
        // Controlador (reglas + operaciones)
        private readonly ProvinciasController _controller;

        public FormProvincias()
        {
            InitializeComponent();

            // 1) Creamos el repositorio con la cadena de conexión
            ProvinciaRepository repo = new ProvinciaRepository(DbConfig.ConnectionString);
```

```
        // 2) Creamos el controlador y le pasamos el repo
        _controller = new ProvinciasController(repo);

        // 3) Ajustes visuales del grid (opcional pero recomendable)
        PrepararGrid();
    }

    private void PrepararGrid()
    {
        // Evita que el usuario agregue filas manualmente en el grid
        dgvProvincias.AllowUserToAddRows = false;

        // Selección de fila completa (más cómodo)
        dgvProvincias.SelectionMode = DataGridViewSelectionMode.FullRowSelect;

        // Solo una fila a la vez
        dgvProvincias.MultiSelect = false;

        // Que se autoajusten las columnas al contenido
        dgvProvincias.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

        // Bloqueamos edición directa en el grid (editaremos con botones)
        dgvProvincias.ReadOnly = true;
    }

    private void FormProvincias_Load(object sender, EventArgs e)
    {
        // Cuando se abre el formulario, cargamos datos
        CargarProvincias();
    }

    private void btnCargar_Click(object sender, EventArgs e)
    {
        // Botón “Cargar” para refrescar manualmente
        CargarProvincias();
    }

    private void CargarProvincias()
    {
        try
        {
            // Pedimos un DataTable al controlador
            DataTable tabla = _controller.ListarTabla();

            // Lo enlazamos al DataGridView
            dgvProvincias.DataSource = tabla;

            // Ajuste de nombres de columnas (opcional)
            if (dgvProvincias.Columns["idprovincia"] != null)
                dgvProvincias.Columns["idprovincia"].HeaderText = "ID";

            if (dgvProvincias.Columns["provincia"] != null)
                dgvProvincias.Columns["provincia"].HeaderText = "Provincia";
        }
        catch (Exception ex)
        {
```

```
        MessageBox.Show("Error al cargar provincias: " + ex.Message);
    }
}

private void dgvProvincias_CellClick(object sender, DataGridViewCellEventArgs e)
{
    // Si el usuario hace clic en una fila, cargamos datos en cajas de texto
    if (e.RowIndex < 0) return; // Clic en cabecera: ignorar

    DataGridViewRow fila = dgvProvincias.Rows[e.RowIndex];

    // Pasamos datos seleccionados a los TextBox
    txtId.Text = Convert.ToString(fila.Cells["idprovincia"].Value);
    txtProvincia.Text = Convert.ToString(fila.Cells["provincia"].Value);
}

private void btnInsertar_Click(object sender, EventArgs e)
{
    try
    {
        // 1) Leemos el nombre del TextBox
        string nombre = txtProvincia.Text;

        // 2) Creamos en BD (el controller valida)
        int nuevoId = _controller.Crear(nombre);

        // 3) Mensaje y refresco del grid
        MessageBox.Show("Insertado correctamente. ID = " + nuevoId);

        CargarProvincias();
        LimpiarCampos();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al insertar: " + ex.Message);
    }
}

private void btnModificar_Click(object sender, EventArgs e)
{
    try
    {
        // 1) Comprobamos que haya un ID seleccionado
        if (string.IsNullOrWhiteSpace(txtId.Text))
        {
            MessageBox.Show("Selecciona una fila para modificar.");
            return;
        }

        // 2) Leemos datos de la pantalla
        int id = Convert.ToInt32(txtId.Text);
        string nombre = txtProvincia.Text;

        // 3) Ejecutamos la modificación
        bool ok = _controller.Modificar(id, nombre);

        if (ok)
```

```
            MessageBox.Show("Modificado correctamente.");
        else
            MessageBox.Show("No se encontró el registro para modificar.");

        CargarProvincias();
        LimpiarCampos();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al modificar: " + ex.Message);
    }
}

private void btnEliminar_Click(object sender, EventArgs e)
{
    try
    {
        // 1) Comprobamos selección
        if (string.IsNullOrWhiteSpace(txtId.Text))
        {
            MessageBox.Show("Selecciona una fila para eliminar.");
            return;
        }

        int id = Convert.ToInt32(txtId.Text);

        // 2) Confirmación (muy recomendable)
        DialogResult r = MessageBox.Show(
            "¿Seguro que quieres eliminar el ID " + id + "?",
            "Confirmar",
            MessageBoxButtons.YesNo,
            MessageBoxIcon.Warning);

        if (r != DialogResult.Yes)
            return;

        // 3) Eliminamos
        bool ok = _controller.Eliminar(id);

        if (ok)
            MessageBox.Show("Eliminado correctamente.");
        else
            MessageBox.Show("No se encontró el registro para eliminar.");

        CargarProvincias();
        LimpiarCampos();
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error al eliminar: " + ex.Message);
    }
}

private void btnLimpiar_Click(object sender, EventArgs e)
{
    LimpiarCampos();
}
```

```
        private void LimpiarCampos()
        {
            // Dejamos la pantalla lista para una nueva inserción
            txtId.Clear();
            txtProvincia.Clear();
            txtProvincia.Focus();
        }
    }
}
```

9. Checklist de eventos a enlazar en el diseñador

Para que funcione, recuerda enlazar estos eventos (doble clic o propiedades -> Events):

- FormProvincias.Load -> FormProvincias_Load
- dgvProvincias.CellClick -> dgvProvincias_CellClick
- btnCargar.Click -> btnCargar_Click
- btnInsertar.Click -> btnInsertar_Click
- btnModificar.Click -> btnModificar_Click
- btnEliminar.Click -> btnEliminar_Click
- btnLimpiar.Click -> btnLimpiar_Click

10. RESUMEN

10.1 Por qué usamos parámetros (@provincia, @id)

- Evitan SQL Injection (seguridad).
- Evitan errores con comillas (por ejemplo O'Connor).
- Mejoran la reutilización del plan de ejecución en SQL Server.

10.2 Por qué usamos using (SqlConnection, SqlCommand…)

- Garantiza que la conexión se cierre aunque haya un error.
- Libera recursos de forma automática (Dispose).
- Evita dejar conexiones abiertas (problema clásico).

10.3 Diferencia entre ExecuteReader, ExecuteScalar y ExecuteNonQuery

- ExecuteReader: devuelve filas (SELECT).
- ExecuteScalar: devuelve un único valor (por ejemplo, el ID insertado).
- ExecuteNonQuery: devuelve cuántas filas se afectaron (INSERT/UPDATE/DELETE).

11. Prácticas para el alumnado (para afianzar)

4. Añade un TextBox de búsqueda y un botón “Buscar” para filtrar por provincia (LIKE).

5. Evita duplicados: antes de insertar, comprueba si ya existe esa provincia.
6. Añade validación: limitar el texto a 100 caracteres y mostrar un mensaje si se pasa.
7. Cambia el grid a edición directa (ReadOnly=false) y crea un botón “Guardar cambios del grid”.

12. Problemas típicos y solución rápida

- No conecta: revisa el Server (., .\SQLEXPRESS o (localdb)\MSSQLLocalDB) y que la BD exista.
- Error de login: si no usas Windows Auth, tendrás que usar User Id=...;Password=...;
- No aparece el diseñador de WinForms: revisa que sea un proyecto WinForms y que compile sin errores.
- La columna no se llama igual: comprueba nombres exactos (idprovincia, provincia).